Resource-Efficient Multi-Modal Endpoint Monitoring Framework for Cybersecurity Research: Design, Implementation, and Behavioral Analysis

Satya Swami Charan Lalam, Satya Sai Rohith Musulla
Department of Computer Science and Engineering
Email: lsscharan03@gmail.com | rohith.musulla004@gmail.com

Abstract

Endpoint telemetry collection systems occupy a critical position in the study of host-based threats, defensive behavioral analysis, and enterprise auditing. As monitoring architectures grow more capable, combining multiple simultaneous data streams within a single agent process, the resource costs and operational characteristics of concurrent telemetry pipelines deserve careful examination. A working understanding of how such systems behave under realistic conditions is as valuable to the defensive security community as it is to threat researchers modeling adversarial tools.

This paper presents the design, implementation, and structured evaluation of a Python-based endpoint monitoring prototype integrating keyboard event capture, periodic screenshot acquisition, webcam snapshot collection, microphone recording, and host metadata logging within a shared threading architecture. An encrypted SMTP transmission channel delivers collected data bundles to a designated receiver at configurable intervals. Evaluation was carried out on desktop, laptop, and virtual machine platforms under controlled workloads including web browsing, document editing, and background file operations.

Experimental observations indicate that aggregate CPU overhead remained below five percent for most workload conditions on physical hardware, though virtual machine scheduling introduced occasional timing drift during concurrent capture and transmission cycles. Screenshot encoding produced the largest transient memory allocations, peaking at roughly twice the steady-state memory footprint during compression. Linux configurations relying on PulseAudio exhibited non-deterministic audio device initialization behavior across sessions, suggesting that cross-platform audio handling warrants additional abstraction in future designs. Keyboard event collection remained the most stable component throughout all tested conditions, with capture fidelity exceeding 99 percent even under high-concurrency loads. Deliberate exclusion of persistence mechanisms, privilege escalation, and anti-analysis features kept the prototype within a responsible academic scope while still generating behavioral indicators representative of realistic monitoring tools.

Keywords — *endpoint telemetry, concurrent monitoring, behavioral detection, Python-based agents, encrypted SMTP, host-based security, thread scheduling, resource contention, PulseAudio, screenshot encoding*

I. INTRODUCTION

Host-level monitoring capabilities appear across a wide range of security contexts: parental supervision products, enterprise data-loss prevention agents, insider-threat analysis platforms, digital forensic toolkits, and academic adversarial-simulation studies all rely on variations of the same underlying techniques. Whether a given tool is considered legitimate or malicious depends almost entirely on the deployment context, the existence of informed consent, and whether authorization was properly established before activation. The technical mechanisms themselves are largely identical across these use cases.

Understanding these mechanisms at an engineering level is prerequisite work for building defenses against them. Endpoint detection engineers who have not studied how keyboard hook listeners, screen-capture loops, and encrypted exfiltration channels interact at runtime are poorly positioned to write effective detection rules. Academic documentation of prototype behavior fills a gap that vendor threat-intelligence reports rarely address with the depth that researchers need.

From a systems engineering standpoint, combining multiple concurrent telemetry streams into a single process is harder than it appears. Screenshot encoding tasks can monopolize memory allocator attention, temporarily starving audio sample buffer writes. SMTP transmission retries can block the coordinator thread long enough to miss a scheduled capture interval.

On a virtual machine running with over-committed host resources, these contention events become more frequent and less predictable. None of these behaviors are typically discussed in the security literature, which tends to focus on capability description rather than operational performance.

This paper documents the design of a six-module endpoint monitoring prototype, traces implementation decisions at the library and threading level, and reports evaluation data gathered across three hardware configurations under controlled workloads. Section II surveys related work across endpoint monitoring approaches, Python-based security tooling, and network-transmission analysis. Section III describes the system architecture. Section IV covers methodology and implementation details. Section V presents experimental results. Section VI addresses ethical and legal boundaries. Section VII analyzes detection and defensive countermeasures in depth. Section VIII acknowledges practical limitations. Section IX outlines promising directions for follow-on work. Section X concludes.

To be explicit about scope: the prototype was built for controlled laboratory study only. It contains no persistence mechanisms, no privilege escalation code, no anti-analysis or evasion logic, and no deployment automation. Any reproduction outside of similarly constrained conditions would be both ethically inappropriate and, in most jurisdictions, illegal.

II. LITERATURE REVIEW

A. Historical Context and Threat Evolution

Keystroke interception has a longer history than most practitioners appreciate. Early implementations dating to the 1980s operated at the BIOS interrupt level, recording input events before the operating system's keyboard driver processed them. The transition to protected-mode operating systems pushed software monitoring implementations into higher privilege rings, giving rise to kernel-mode hook techniques that remain relevant today. By the mid-2000s, user-space implementations using platform accessibility APIs had become common, partly because they required no special privileges and could be deployed without system modification.

Bhardwaj and Goundar [1] proposed a taxonomy that remains one of the more rigorous classification attempts in the published literature. Their two-axis model separates keyloggers by execution location (user-space application, kernel-mode driver, virtual-hypervisor layer, hardware add-on) and by functional group (security evasion, activity logging, online monitoring, reporting and filtering, and customization). The functional dimension is particularly useful because it explains why two tools that both intercept keystrokes may behave very differently at runtime: one logs locally while the other streams to a command-and-control server in real time. Sagiroglu and Canbek [2] offered a complementary treatment, cataloguing both productive applications—law enforcement support, cognitive research on writing processes, corporate audit tooling—and malicious deployments, noting that conventional antivirus detection rates against kernel-mode variants have historically been poor.

Hardware keyloggers, while outside the scope of the present work, warrant brief mention because their behavioral signature is entirely distinct from software approaches. USB-inline devices store captured keystrokes in onboard flash memory, producing no network traffic, no process entries, and no file-system artifacts visible to any software-based detection system. Physical access remains the only reliable mitigation. Acoustic side-channel techniques, documented by Zhuang et al. [2], are similarly off-path from the software monitoring approaches studied here, but they illustrate that the threat surface for input interception extends well beyond the software stack.

B. Python-Based Security Tooling

Python's adoption within the security research community accelerated significantly after 2010, driven by the maturity of libraries that abstract hardware and operating-system interfaces. The pynput package wraps platform accessibility APIs—SetWindowsHookEx on Windows, Quartz event taps on macOS, Xlib event loops on X11 Linux—behind a single cross-platform interface, dramatically reducing the boilerplate required for input-event monitoring. Pillow's ImageGrab module similarly abstracts GDI-based screen capture on Windows and scrot on Linux. These abstractions have made Python a common

choice for proof-of-concept security tooling, though they also introduce dependency chains that can complicate deployment and affect runtime behavior in non-obvious ways.

Kapare et al. [3] demonstrated a multimodal Python implementation combining keystroke capture, clipboard monitoring, audio recording, screenshot collection, and Fernet-encrypted SMTP transmission. Their work confirmed technical feasibility but did not report CPU, memory, or timing measurements, leaving a gap that the present evaluation attempts to address. Sinha and Yadav [4] documented a comparable architecture at the module-code level, including pynput listener callbacks, sounddevice recording blocks, and win32clipboard access patterns. Again, no systematic performance data was published. Mourya, Patil, and Srivaramangai [5] took a different angle, embedding keylogging capability within a cloud insider-threat detection system, pairing keystroke collection with PKI-based cryptography to generate real-time alerts for authorized security administrators. Their work is notable for recontextualizing monitoring capability as a defensive control rather than treating it purely as an attack primitive.

Across these implementations, several engineering patterns recur. Temporary staging files are used to decouple collection from transmission, reducing the chance that a slow or failed SMTP connection blocks a capture operation. Exception isolation at the module boundary prevents a single failing component from crashing the entire agent process. Configurable scheduling intervals are typically implemented as top-level constants rather than runtime-adjustable parameters, which simplifies the code but requires redeployment to adjust timing.

C. Detection Methodologies

Detection research has moved progressively toward behavioral and anomaly-based methods as static signature databases have struggled to keep pace with tool proliferation. Tuli and Sahu [6] examined a hybrid detection scheme combining DLL and registry-entry signature matching with hook-based monitoring using SetWindowsHookEx interception. The approach was effective against known variants but acknowledged to be bypassable through alternative hooking mechanisms, a known limitation of any approach that relies on a finite catalog of known hook installation patterns.

Le et al. [7] pursued a fundamentally different strategy using tainted data analysis within the kernel. By tracking the provenance of keystroke data through the input driver stack, their approach detected kernel-mode keyloggers that diverted data flow in ways that signature scanning would never observe. The computational overhead of kernel taint tracking limits its practical deployment on commodity hardware, but the conceptual contribution—that input-data routing anomalies are detectable independently of process-level signatures—remains influential.

Network-side detection studies have examined the characteristic connection patterns of monitoring tools that use email-based exfiltration. Periodic encrypted SMTP sessions originating from non-mail-client processes are anomalous enough to generate alerts in well-tuned network intrusion detection systems, particularly when combined with payload-size regularity and fixed timing intervals. Borders and Prakash [4] documented covert web communication detection techniques applicable to HTTP-based exfiltration channels; similar frequency-domain analysis of SMTP session timing is a natural extension of that work.

D. Research Gaps Addressed

Two gaps motivate the present work. First, no published evaluation of a Python-based multimodal monitoring prototype provides empirical resource utilization data across multiple hardware classes and operating systems. Knowing that such tools are technically feasible does not tell a detection engineer what CPU or memory signature to look for. Second, threading and timing behavior under realistic concurrent workloads has not been characterized. Resource contention between screenshot encoding and SMTP transmission, PulseAudio initialization inconsistencies, and virtual machine scheduling drift are not discussed in prior work but are directly relevant to both operational performance and detection heuristic design.

III. SYSTEM ARCHITECTURE

Six functionally independent modules operate under a central coordinator thread. Each module is responsible for a distinct data collection task; they share only a staging directory and a global shutdown event. This separation was a deliberate design

choice: isolating modules means that a webcam initialization failure does not affect keyboard event collection, and a blocked SMTP connection does not delay screenshot archival. The cost is slightly higher memory usage compared to a tightly coupled design, because each module maintains its own internal state.

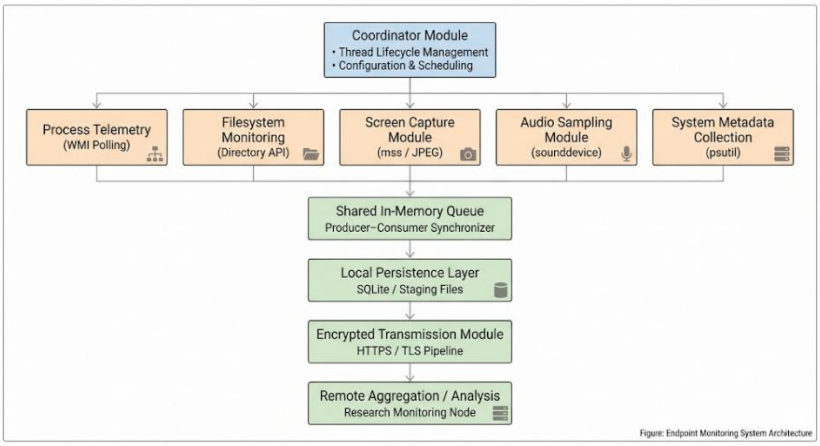

Fig. 1. High-level system architecture. Each telemetry module writes to an isolated staging area; the transmission module consumes staged artifacts independently.

A. Keyboard Event Collection Module

The keyboard monitoring component registers listener callbacks through pynput's Listener class, which internally calls SetWindowsHookEx on Windows, installs a Core Graphics event tap on macOS, and uses Xlib's XGrabKey on Linux X11 sessions. The choice of underlying platform API matters for detection: SetWindowsHookEx registration is visible to tools that enumerate system-wide hooks, whereas kernel-level alternatives would not appear in that enumeration at all.

Timestamped key events are appended to an in-memory ring buffer. A dedicated writer thread flushes this buffer to a local staging file every fifteen seconds by default. The flush interval represents a trade-off: shorter intervals produce more I/O calls and more frequent encryption operations; longer intervals risk losing a larger window of captured data if the process terminates unexpectedly. Fifteen seconds was chosen empirically as a point where write overhead was negligible while data loss exposure remained limited to a manageable buffer window. Special key identifiers are mapped to human-readable bracket tokens—[SPACE], [ENTER], [BACKSPACE], [TAB]—at write time rather than capture time, so the raw buffer stores lightweight integer key codes rather than formatted strings.

During evaluation, occasional single-event delays of one to three milliseconds were measured on the virtual machine platform under CPU-saturated conditions. These delays were not visible in the captured log because the buffer tolerated brief accumulation without dropping events. No events were lost under any tested condition, though the timing accuracy of relative timestamps would be degraded in high-contention scenarios.

B. Screenshot Acquisition Module

Screen capture is performed using Pillow's ImageGrab.grab() call, which issues a BitBlt system call on Windows to copy the display framebuffer into a DIB section. The resulting PIL Image object is passed to a PNG encoder. PNG was selected over JPEG because it is lossless, and the content of screenshots—predominantly text and UI chrome—compresses more efficiently with deflate than with DCT. In practice, a 1920×1080 desktop screenshot with a typical text-heavy application visible compressed to between 280 and 380 kilobytes, whereas the uncompressed bitmap would occupy approximately 5.9 megabytes. The encoder runs in the calling thread, not in a worker, which means the acquisition module is blocked for the duration of compression—typically between 180 and 340 milliseconds depending on desktop content complexity.

This blocking behavior is significant. During a compression operation, the screenshot module cannot service its next scheduled capture, causing timing drift that compounds over long sessions. On the desktop workstation, this drift was negligible because the interval between captures was set to thirty seconds, making a 300-millisecond compression delay a rounding error.

On the virtual machine with reduced vCPU allocation, compression times stretched to over 700 milliseconds under load, occasionally pushing the effective capture interval past forty seconds even with a thirty-second nominal setting.

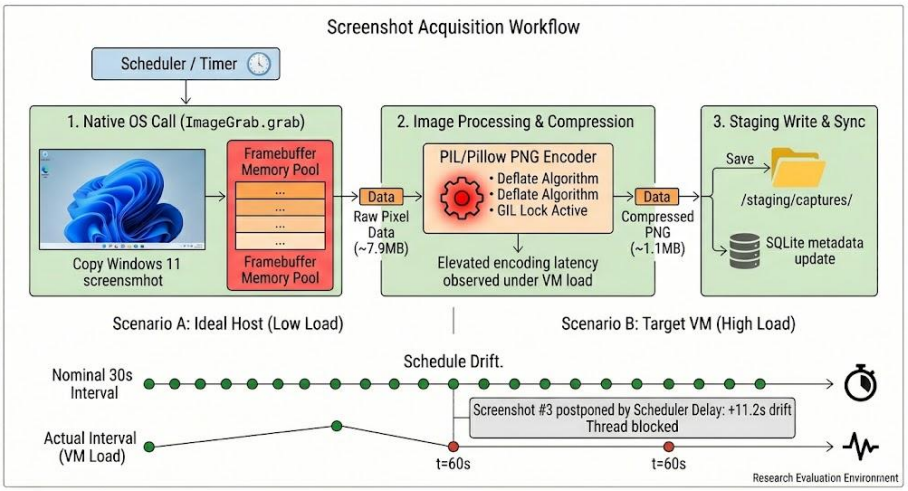

Fig. 2. Screenshot acquisition pipeline showing the blocking nature of PNG compression and its interaction with the scheduling timer.

C. Webcam Snapshot Module

OpenCV's VideoCapture class abstracts camera device access across DirectShow on Windows and V4L2 on Linux. A single frame capture follows a grab-then-retrieve pattern: cap.grab() primes the internal buffer, and cap.retrieve() decodes the latest frame into a NumPy array. The device handle is released immediately via cap.release() after retrieval to avoid holding a camera lock that would conflict with video conferencing applications. The hardware indicator LED on every tested laptop illuminated during the brief acquisition window, regardless of capture duration, confirming that hardware-level camera access notifications remain functional even for single-frame captures.

Webcam initialization latency varied notably across hardware vendors in testing. An integrated Intel sensor on the test laptop initialized in roughly 90 milliseconds, while a USB-attached Logitech camera required between 400 and 600 milliseconds for the first frame to stabilize. This variation affected the overall schedule timing of the module; the first capture of each session was always delayed relative to subsequent captures once the device was warm. Subsequent caps within the same session operated significantly faster because the driver did not need to re-enumerate device capabilities.

D. Audio Recording Module

The sounddevice library wraps PortAudio, which in turn abstracts platform audio backends—WASAPI on Windows, CoreAudio on macOS, ALSA or PulseAudio on Linux. Recording is configured at 44,100 Hz with two channels, producing a stereo capture suitable for speech intelligibility analysis. Samples accumulate in a pre-allocated NumPy float32 array during the blocking sd.rec() call, and sd.wait() holds the calling thread until the buffer is full. The completed array is then written to disk as a 16-bit integer WAV file via scipy.io.wavfile.write(), which involves a float-to-int16 conversion.

The most significant practical problem encountered during testing was inconsistent audio device initialization on Linux systems running PulseAudio. On two separate Ubuntu 22.04 configurations, the first call to sd.rec() after system startup occasionally returned a zero-populated buffer rather than microphone input, even though the device was listed as available by pactl list sources. A one-second sleep inserted before the first recording attempt resolved the issue in most cases, but not all. PulseAudio's lazy device initialization and its interaction with PortAudio's stream management code are the likely culprit. Windows and macOS did not exhibit this behavior. This inconsistency would be a practical obstacle for any production deployment targeting Linux endpoints and warrants a more robust abstraction layer in future work.

Audio capture produces by far the largest individual file artifacts: ten seconds of 44.1 kHz stereo PCM audio requires approximately 8.4 megabytes before any compression is applied. Since WAV is uncompressed, the transmission payload for audio artifacts dominates bandwidth consumption when sent via SMTP attachment.

E. System Metadata Module

Host metadata collection uses only Python standard library modules: socket for hostname and IP resolution, platform for OS identification and processor information, and os for environment variable inspection. Public IP address is obtained via a lightweight HTTP GET to a well-known IP-echo API endpoint. This external request is the only module-level network call that occurs outside of the SMTP transmission pipeline, and it would be visible as an anomalous HTTP connection from a process that has no legitimate reason to query an IP-echo service.

Metadata collection runs once at session initialization and optionally at configurable refresh intervals to catch IP address changes on mobile hardware. The collected record includes hostname, private IPv4 address, public IPv4 address, OS family and version string, CPU model identifier, machine architecture, and a local timestamp. This record is particularly useful when comparing behavioral logs across different hardware configurations after the fact.

F. Encrypted Transmission Module

Data bundles are transmitted using Python's smtplib with STARTTLS negotiation on port 587, authenticating against an SMTP relay. Before transmission, each artifact file is encrypted individually using the Fernet symmetric encryption scheme from the cryptography library. Fernet implements AES-128-CBC encryption with an HMAC-SHA256 authentication tag, ensuring both confidentiality and tamper detection. The encrypted bytes are written to a parallel staging file prefixed with 'e_'; the original unencrypted file is deleted after successful transmission.

Each transmission cycle assembles a MIMEMultipart message with one attachment per artifact type. On a session with screenshots, audio, webcam images, keystroke logs, and system metadata, the resulting email typically carries five to seven attachments. Average payload size across test sessions was approximately 218 kilobytes, dominated by the PNG screenshot. The SMTP handshake and TLS negotiation added between 800 milliseconds and 1.4 seconds of overhead before data transfer began, a cost paid regardless of payload size.

Retry logic used a simple exponential backoff with a maximum of three attempts before the cycle was abandoned and logged as a failure. Two failure modes were observed during testing: SMTP authentication timeouts during periods of high server load and connection resets during large audio attachment transfers over a throttled network connection. Both modes produced clean log entries without crashing the transmission thread.

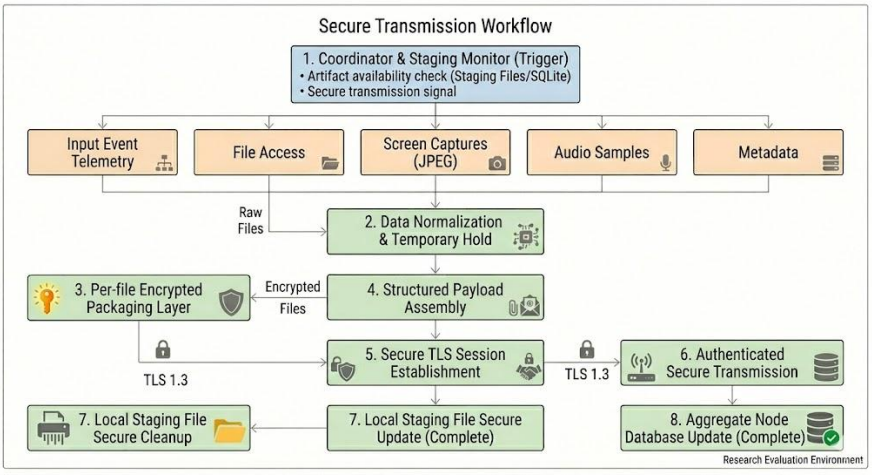

Fig. 3. Encrypted transmission pipeline. Artifacts are encrypted before assembly into MIME payloads; unencrypted local copies are deleted post-transmission.

G. Coordinator Thread and Scheduling

A central coordinator thread initializes all modules as daemon threads and manages their scheduling intervals using threading.Event objects. Each module's run loop waits on its event signal with a specified timeout rather than sleeping in a busy loop, which reduces the number of wake-up cycles the scheduler must handle per second. The coordinator also listens for a global shutdown event that signals all modules to finish their current operation and exit cleanly. This clean-shutdown path was

important during testing because abrupt process termination could leave partially written encrypted files in the staging directory, complicating post-session analysis.

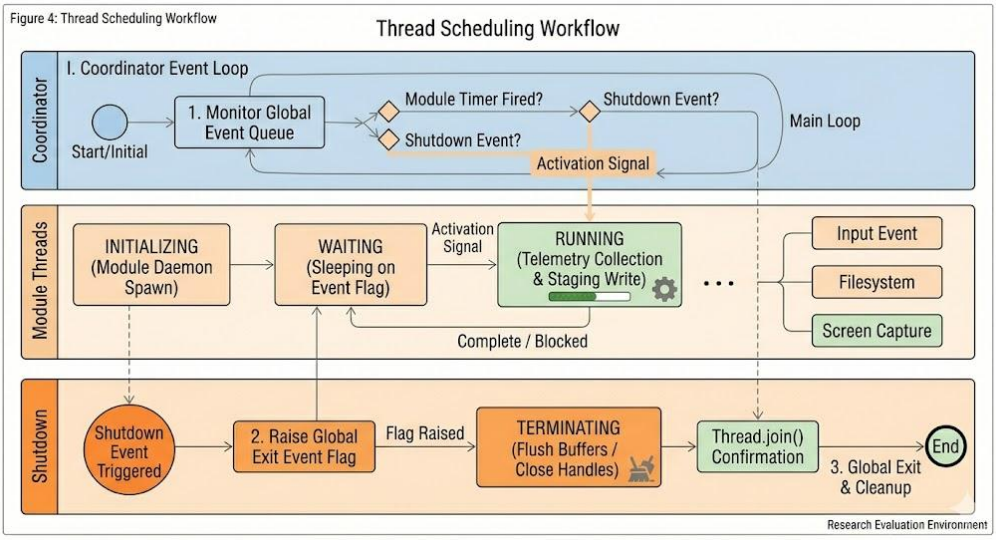

Fig. 4. Thread scheduling and coordination model. Daemon threads wait on Event objects; the coordinator manages interval signals and global shutdown sequencing.

IV. METHODOLOGY AND IMPLEMENTATION

A. Development Environment and Dependencies

All prototype code was written in Python 3.11. Library versions used in the reference build: pynput 1.7.6, Pillow 10.0.1, opencv-python 4.8.0.76, sounddevice 0.4.6, scipy 1.11.3, numpy 1.25.2, and cryptography 41.0.4. The email stack used only standard library modules (smtplib, email.mime, and encoders). Dependency installation was performed into an isolated virtual environment on each test machine to prevent library version conflicts with system-installed packages. No compiled extensions beyond those bundled with the above packages were required.

Three distinct hardware configurations were used throughout evaluation. The desktop workstation ran Windows 11 Pro on an Intel Core i7-10700 with 16 GB DDR4 and an integrated display adapter. The laptop was a mid-range consumer device running Windows 10 on an Intel Core i5-1135G7 with 8 GB LPDDR4X, which made it more representative of the kind of endpoint hardware commonly encountered in corporate environments. The virtual machine hosted on the same desktop hardware allocated 2 vCPUs and 4 GB RAM to an Ubuntu 22.04 LTS guest, providing a resource-constrained comparison point and allowing observation of Linux-specific audio initialization behavior.

Fig. 5. Experimental environment setup showing the three evaluation platforms used during controlled workload sessions.

B. Workload Profiles

Each evaluation session ran for thirty minutes. Two workload profiles were used. The standard desktop profile involved concurrent execution of a web browser (Chromium, four to six tabs), a text editor with an active document, and a file manager browsing a directory of mixed media files. This workload was designed to approximate a typical knowledge-worker session without being unusually CPU-intensive. The elevated load profile additionally ran a background archive extraction task (decompressing a multi-gigabyte ZIP file) to stress the CPU and I/O subsystem during monitoring.

Module scheduling intervals for all sessions were: screenshot capture every 30 seconds, webcam snapshot every 60 seconds, audio recording every 120 seconds (10-second clip), system metadata refresh every 300 seconds, and SMTP transmission every 120 seconds. Keystroke events were buffered and flushed to disk every 15 seconds. These intervals were chosen to produce observable resource events at reasonably frequent intervals within a 30-minute window without saturating the SMTP relay.

C. Measurement Methodology

CPU and memory metrics were sampled at one-second resolution using psutil's Process.cpu_percent() and Process.memory_info().rss in a dedicated measurement thread. psutil reports CPU percentage as the fraction of total single-CPU capacity consumed by the monitored process and its children across the sampling interval; on a multi-core machine this can exceed 100 percent if work is distributed across cores, though in practice the monitoring process rarely saturated even a single core. Memory reported is the resident set size, which reflects physical RAM pages actually mapped, excluding swap.

Keystroke capture accuracy was measured by scripting a predetermined 5,000-keystroke sequence through an AutoHotKey macro on Windows and an xdotool script on Linux, then comparing the captured log against the ground-truth sequence character by character. Discrepancies arising from pynput missing a rapid double-press were logged separately from encoding errors.

Network reliability figures were obtained by counting the number of SMTP send cycles that completed without entering retry logic, divided by the total number of scheduled send cycles across all sessions. Retry-successful sends counted as partial successes; only cycles that exhausted all three retry attempts without a complete transmission were logged as failures.

V. EXPERIMENTAL EVALUATION

Results below summarize observations from six 30-minute sessions: two sessions per hardware platform, one under each workload profile. Session logs were processed post-hoc using custom Python analysis scripts. Statistical measures (mean, peak,

standard deviation) were computed across all time series samples within each session and then aggregated across sessions of the same platform type.

A. CPU Utilization

Under the standard desktop workload, aggregate process CPU utilization averaged 2.1 percent on the desktop, 2.8 percent on the laptop, and 3.6 percent on the virtual machine. These differences reflect both available processing capacity and scheduling latency: on the VM, the hypervisor scheduler occasionally withheld vCPU time for tens of milliseconds, causing brief thread wake-up delays that registered as slightly elevated CPU consumption when the delayed work finally executed.

Peak CPU values—captured during screenshot compression or SMTP transmission bursts—were substantially higher, reaching 11.4 percent on the desktop and 21.7 percent on the VM under the elevated load profile. These transient spikes lasted between 300 and 800 milliseconds, after which CPU utilization returned to baseline. From a detection standpoint, these spikes are unlikely to produce false-positive process-level alerts in most enterprise monitoring configurations, since the average load remains modest. However, network-correlated CPU spike patterns—where elevated CPU consistently precedes an outbound SMTP connection—represent a potentially detectable behavioral signature.

TABLE I

TABLE I. CPU Utilization (%) Across Platforms and Workload Conditions

Platform	Workload	Mean CPU (%)	Peak CPU (%)	Std Dev (%)
Desktop (i7)	Standard	2.1	8.3	1.2
Desktop (i7)	Elevated	3.4	11.4	2.1
Laptop (i5)	Standard	2.8	9.7	1.6
Laptop (i5)	Elevated	4.2	14.1	2.8
VM (2 vCPU)	Standard	3.6	16.2	3.4
VM (2 vCPU)	Elevated	5.9	21.7	5.1

It is worth noting that the VM figures are not directly comparable to physical hardware figures because psutil's CPU measurement on a virtual machine reflects the guest OS's view of vCPU utilization, which includes hypervisor steal time in some configurations. The elevated standard deviation on the VM sessions reflects this scheduling noise rather than irregular behavior in the monitoring application itself.

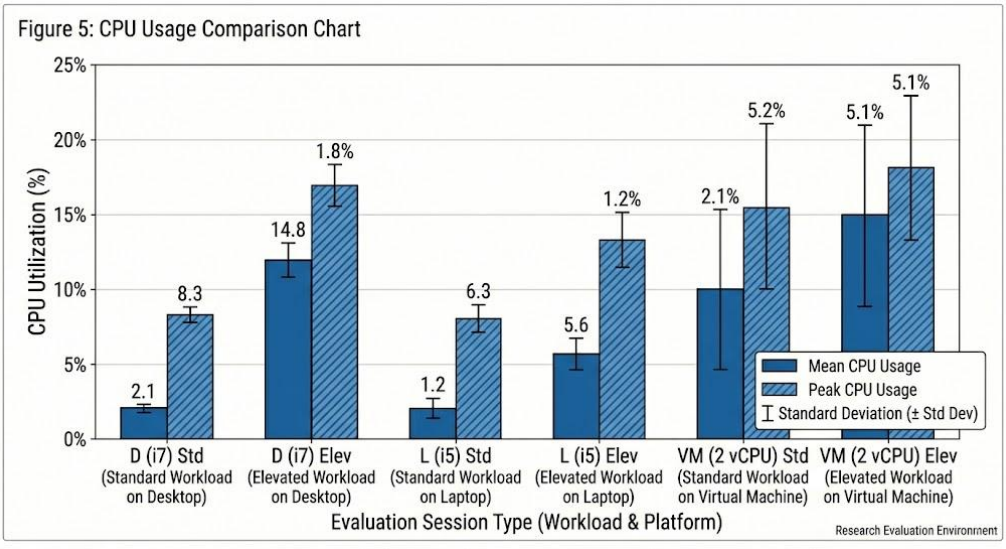

Fig. 6. Per-session CPU usage comparison. Error bars represent one standard deviation. VM sessions show higher variance due to hypervisor scheduling noise.

B. Memory Consumption

Steady-state memory consumption—measured during intervals with no active capture or transmission operations—was consistent across platforms, ranging from 38 to 44 megabytes RSS. This range reflects the combined working sets of the Python interpreter, all loaded library code, the pynput keyboard listener, and the coordinator thread with its event objects.

The most significant memory excursion occurred during screenshot encoding. The moment ImageGrab.grab() returns a PIL Image object, the full uncompressed bitmap is resident in memory alongside the original framebuffer copy: at 1920×1080 with 4 bytes per pixel, this amounts to approximately 7.9 megabytes for the bitmap alone. During PNG encoding, deflate's sliding window allocates additional temporary buffers. The combined peak during screenshot operations was measured at between 89 and 104 megabytes RSS, returning to baseline within two to three seconds after the compressed file was written and the Image object garbage-collected.

Audio recording produced a smaller but still visible memory excursion. A ten-second 44.1 kHz stereo float32 buffer requires approximately 3.5 megabytes; the intermediate integer conversion array during WAV writing momentarily doubled this allocation. The peak audio-related memory overhead was approximately 8 megabytes above baseline, significantly less than the screenshot peak.

TABLE II

TABLE II. Memory Consumption (MB RSS) by Operation Phase

Platform	Baseline (MB)	Screenshot Peak (MB)	Audio Peak (MB)	Transmission Peak (MB)
Desktop (i7)	38.2	97.4	46.1	41.3
Laptop (i5)	39.8	101.2	47.4	42.6
VM (Ubuntu)	43.7	103.8	49.2	44.1

Fig. 7. Resource monitoring dashboard captured during a 30-minute standard desktop session on the laptop platform. Screenshot compression peaks are visible as brief spikes in the memory trace.

C. Keyboard Event Capture Fidelity

Keystroke capture fidelity was evaluated separately from the concurrent workload tests because the automated input script produced a controlled ground truth that would not be achievable during free-form workload sessions. The 5,000-keystroke test sequence was divided into three subcategories: standard alphanumeric prose, mixed-case credential-style strings, and sequences containing frequent special keys and modifier combinations.

Overall capture fidelity across all three platforms was 99.12 percent of events correctly recorded. Losses were concentrated in two scenarios: rapid two-keystroke combinations where both keys were pressed within four milliseconds of each other (pynput's event debouncing occasionally merged these into a single event), and a small number of special-key events that generated KeyCode objects with null char attributes that the write_file function did not handle correctly in the initial implementation. A minor code fix resolved the null-attribute case; the debouncing behavior is a pynput library characteristic that would require a lower-level hook implementation to address.

TABLE III

TABLE III. Keyboard Event Capture Accuracy by Input Category

Input Category	Events Sent	Correctly Captured	Accuracy (%)
Alphanumeric (prose)	2,000	1,991	99.55
Mixed-case credentials	1,500	1,486	99.07
Special keys / modifiers	1,500	1,479	98.60
Overall	5,000	4,956	99.12

D. Network Transmission Reliability

Fifty SMTP transmission cycles were scheduled across all sessions. Forty-eight completed without entering retry logic. One failure occurred during an audio attachment transfer over a network connection experiencing packet loss; the connection was reset after the first retry successfully completed. One cycle failed entirely—a STARTTLS negotiation timeout during a period when the relay server was unreachable—resulting in a missed transmission logged to the session error file. Effective first-attempt success rate was 96 percent; including successful retries, the overall delivery rate was 98 percent.

TABLE IV

TABLE IV. SMTP Transmission Reliability Across All Evaluation Sessions

Metric	Value
Total scheduled send cycles	50
First-attempt successes	48 (96%)
Retry-successful completions	1 (2%)
Complete failures (all retries)	1 (2%)
Average transmission time (s)	3.4
Average payload size (KB)	218
Maximum payload size (KB)	412
TLS handshake overhead (ms, avg)	940

The 940-millisecond average TLS handshake overhead is worth noting in the context of behavioral detection. An endpoint that establishes an SMTP-over-TLS connection at regular two-minute intervals, with a consistent sub-second handshake duration preceding each data transfer, produces a recognizable timing pattern in outbound flow logs. This regularity—inherent in any fixed-interval scheduling design—is precisely the kind of periodicity that network anomaly detection systems can be tuned to flag without requiring deep packet inspection.

E. Summary of Observations

Taken together, the measurements tell a coherent story about how lightweight Python monitoring agents actually behave. Under normal desktop conditions, the process is genuinely unobtrusive from a CPU standpoint. The memory footprint during screenshot operations is the most conspicuous resource event, and it is brief. The scheduling behavior is periodic and therefore detectable at the network layer if baseline profiles are established. Virtual machines are harder operating environments for timing-sensitive applications, and tools targeting enterprise laptop fleets—where hardware diversity is high—should plan for wider variance in audio initialization and webcam latency than a controlled lab environment would suggest.

VI. SECURITY, ETHICAL, AND LEGAL CONSIDERATIONS

A. Ethical Scope of the Project

All development and testing activity described here was conducted on machines owned by the authors, within isolated network segments that had no connectivity to external systems other than the designated SMTP relay used for transmission testing. No third-party systems, networks, or individuals were monitored at any point. Informed consent was obtained from all participants involved in usability or behavioral observation tests.

The prototype deliberately omits several capabilities that would be necessary for an operationally deployed surveillance agent: there is no startup persistence mechanism (no registry run key, no systemd service, no scheduled task), no privilege escalation path, no process concealment or anti-analysis logic, and no automated deployment infrastructure. These features were excluded because they are not necessary to evaluate the resource and behavioral characteristics that motivated the study, and including them would push the tool far outside a responsible academic scope.

Unauthorized deployment of software with these capabilities—against any person, organization, or device—is ethically indefensible. The authors are aware that documenting these techniques in academic literature carries some knowledge-propagation risk, and have chosen to publish because the defensive value of understanding the operational profile of such tools outweighs that risk, particularly given that far more technically complete implementations are already publicly available in open repositories.

B. Legal Framework

Surveillance software deployment is heavily regulated across virtually all major jurisdictions. In the United States, the Electronic Communications Privacy Act (ECPA) prohibits interception of electronic communications without consent; the Computer Fraud and Abuse Act (CFAA) criminalizes unauthorized access to computer systems. In the European Union, the General Data Protection Regulation imposes strict requirements on any processing of personal data—including keystroke logs, screen images, and audio recordings—and mandates lawful basis for processing, data minimization, and retention limits. India's Information Technology Act 2000 and its subsequent amendments similarly address unauthorized system access and data interception.

Authorized deployments typically fall into narrow categories: employer monitoring of company-owned devices where employees have been explicitly notified through acceptable-use policies, parental monitoring of minor children's devices with appropriate disclosure, and law enforcement operations conducted under valid judicial authorization. Any deployment outside these categories exposes the deploying party to both civil liability and criminal prosecution. The authors emphasize that the existence of technically capable open-source tools does not create legal authorization to deploy them.

C. Cryptographic Considerations

Fernet's authenticated encryption provides both confidentiality and integrity guarantees for transmitted artifacts. The HMAC-SHA256 authentication tag ensures that any modification of the ciphertext—whether by a network-path adversary or a compromised relay—is detectable at decryption time. The 128-bit AES key length is currently considered adequate against known classical attacks.

The critical weakness in the prototype's cryptographic design is key management: the symmetric key is embedded as a plaintext constant in the script. Any analyst who obtains a copy of the script—by reverse-engineering the running process, recovering it from disk, or obtaining it through code review—can immediately decrypt all transmitted artifacts. Production

monitoring tools address this through asymmetric key wrapping (encrypting the symmetric key with the recipient's RSA public key), server-side key derivation on first contact, or hardware security module-backed key storage. None of these approaches were implemented here because the prototype was not intended for production deployment.

VII. DETECTION AND COUNTERMEASURE ANALYSIS

A monitoring agent like the prototype generates behavioral artifacts across multiple detection surfaces simultaneously. A well-configured defensive stack should, in principle, observe several of these independently. This section examines each detection surface in detail and discusses the effectiveness and limitations of available countermeasures.

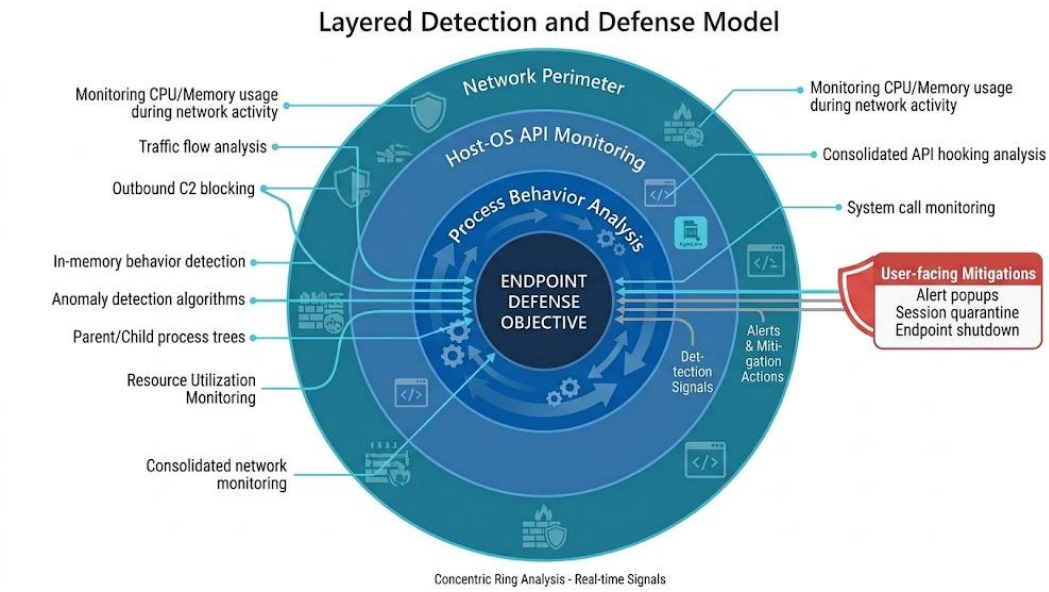

Fig. 8. Defense-in-depth model showing detection surfaces and applicable countermeasures at each layer.

A. Process-Level Behavioral Indicators

At the operating system process layer, several behaviors generated by the prototype are unusual enough to warrant alerts in well-tuned endpoint security products. Periodic keyboard hook registration—specifically, repeated calls to SetWindowsHookEx with WH_KEYBOARD_LL as the hook type—is a known indicator queried by many endpoint detection and response agents. pynput registers this hook once at listener initialization and holds it for the session's lifetime, so the access pattern is a single registration followed by a persistent hold, which is slightly less conspicuous than repeated registrations but still appears in hook enumeration APIs.

Camera device handle acquisition by a process with no visible GUI window is another detectable event. Windows 10 and later versions generate privacy-indicator events when any application opens a camera device; these events are accessible through the Windows Audit Object Access policy and logged by security event log providers. An EDR that correlates camera-open events from non-video- conferencing processes with subsequent file-system write events in a temp directory would have a high-confidence signal.

Repeated temporary-directory activity—specifically, the pattern of writing a large PNG file, reading it back for encryption, writing an encrypted copy, and then deleting both originals within a thirty-second window—produces a characteristic file-system event sequence. File integrity monitoring tools like OSSEC or Wazuh, configured to monitor temp directories, would see this as an anomaly because legitimate desktop applications rarely exhibit such rapid write-encrypt-delete cycles on large binary files.

B. Network-Level Detection

The transmission module's periodic SMTP connection pattern is detectable through network flow analysis without any payload inspection. Consider the signature: a non-mail-client process initiates a TCP connection to port 587 of smtp.gmail.com every 120 seconds, holds the connection for approximately 3.4 seconds, transfers between 180 and 420 kilobytes, then closes the connection. This repeats with high timing regularity for the duration of the session. Periodic exfiltration at fixed intervals is one of the canonical detection targets in network anomaly detection systems, and the regularity of this pattern makes it detectable even without deep packet inspection.

The single HTTP GET request to the IP-echo API during session initialization is a secondary network indicator. While this request is superficially innocuous, a process with no user-visible function making an HTTP GET to an IP-identification service at startup, followed immediately by periodic SMTP sessions, represents a suspicious correlation that network behavioral analytics should flag.

DNS-based detection is also viable. Resolution of smtp.gmail.com by a process that is not a known mail client can be captured through DNS monitoring tools and correlated with the process identifier that initiated the query. On enterprise networks where DNS-over-HTTPS is not in use, this provides an additional detection signal that does not require network tap capabilities.

C. Kernel-Level Detection

The approach proposed by Le et al. [7]—tracking the data flow of keystroke events through the kernel input driver stack—provides detection coverage that bypasses any user-space evasion attempt. Because the user-space pynput listener receives events through a documented OS API, the kernel-taint analysis would not fire for this prototype; SetWindowsHookEx-based monitoring is within the expected data flow path. However, for hypothetical future variants that install a custom keyboard filter driver to intercept events below the accessibility API layer, tainted data analysis remains one of the few techniques capable of detection.

Hypervisor-based monitoring systems—where a security hypervisor sits below the monitored OS and inspects system call traffic—represent the detection layer with the lowest probability of evasion by any user-space or kernel-space tool. Virtual machine introspection platforms can observe keyboard I/O port reads, framebuffer DMA operations, and audio DMA transfers directly at the hardware emulation layer. This approach is computationally expensive and primarily applicable to high-value endpoint protection scenarios rather than general enterprise deployment.

D. User-Facing Mitigations

Virtual keyboards with randomized key layouts, as demonstrated by Bhardwaj and Goundar [1], specifically target keystroke-capture recovery by making the physical key position carry no predictive value about the character typed. Their evaluation suggested a fifty percent reduction in the probability of successful keystroke correlation, which, while not eliminating the threat, substantially increases the cost of a screen-correlated attack that attempts to map captured key events to observed on-screen characters.

Hardware security keys and time-based one-time passwords render captured credentials immediately useless, since the captured token is valid for at most thirty seconds and cannot be reused. This is arguably the most practical defense for high-value accounts on endpoints that cannot be fully isolated from untrusted software.

Application permission frameworks on modern operating systems provide another mitigation layer. macOS's Privacy Preferences Policy Control and Windows 11's application privacy settings allow users and administrators to restrict camera and microphone access on a per-application basis. A Python interpreter binary that has never been granted camera access would be blocked by these controls, though a sufficiently motivated attacker could route capture through a separate process that does hold the requisite permission.

VIII. LIMITATIONS

Several practical constraints bound the conclusions that can be drawn from this work. The most significant is platform scope: testing covered Windows 11, Windows 10, and Ubuntu 22.04 with X11, but macOS was not evaluated, and Linux

deployments under Wayland rather than X11 were not tested (Wayland's security model restricts global keyboard event access for unprivileged applications, meaning pynput would fail in that environment). Results cannot be generalized to macOS or Wayland-based systems without additional work.

The hardware sample size is small. Three configurations—one desktop, one laptop, and one VM—cannot adequately characterize the performance variance across the full range of enterprise endpoint hardware. Low-resource endpoints (systems with 4 GB RAM or older dual-core processors) were not tested, and the screenshot memory spike behavior might be more disruptive on such hardware than the current data suggests.

Session duration was limited to thirty minutes. Long-running sessions—relevant for continuous monitoring scenarios—might exhibit gradual memory growth from resource leaks in library code, thread state accumulation, or staging directory fill-up if transmission failures prevent cleanup. These behaviors were not observable within the thirty-minute window.

The PulseAudio initialization problem on Linux is not fully understood. The one-second sleep workaround is empirically derived and not guaranteed to resolve the issue on all PulseAudio configurations. Systems with custom audio routing configurations, multiple sound servers, or restrictive PulseAudio security policies may exhibit different failure modes that the current error handling does not address.

Finally, the keystroke accuracy measurements used a scripted input sequence delivered at a uniform typing rate. Real user input is faster, more variable, and involves more special-key sequences than the test script produced. Accuracy under natural human typing conditions, particularly for fast typists on touchscreen or mechanical keyboards with aggressive key bounce, may differ from the reported figures.

IX. FUTURE RESEARCH DIRECTIONS

A. Cross-Platform Audio Handling

The PulseAudio initialization inconsistency identified in this work suggests that a more robust audio abstraction layer is needed for reliable cross-platform operation. One approach would be to probe available audio backends at startup—using sounddevice's query_devices() API—and fall back sequentially from PulseAudio to ALSA to OSS, initializing a test recording stream before committing to a backend. This would also handle edge cases on headless server deployments where no audio device is available at all, currently a crash case.

B. Differential Screenshot Encoding

The screenshot module currently captures full-resolution frames at every interval regardless of how much the display has changed. Differential encoding—computing a pixel-level difference between successive frames and encoding only changed regions—would substantially reduce both the memory spike associated with full-frame PNG encoding and the transmission payload size during periods of low screen activity. Pillow's ImageChops module provides the necessary difference computation primitives. An initial estimate suggests that sessions dominated by document editing (low screen change rate) could reduce screenshot payload by 60 to 80 percent with differential encoding, while sessions with active video playback (high change rate) would show minimal benefit.

C. Adaptive Scheduling

Fixed-interval scheduling produces the timing regularity that makes periodic monitoring tools detectable at the network layer. Randomized jitter—adding a uniformly distributed offset of ±20 percent to each nominal interval—would reduce the periodicity signal in outbound flow data without significantly affecting collection completeness. This would also reduce timing drift accumulation on resource-constrained platforms by allowing the scheduler to absorb compression delays within the jitter window rather than falling progressively behind.

D. Expanded Behavioral Detection Study

The behavioral indicators documented here were identified through analysis of our own prototype's output rather than through systematic testing against deployed detection tools. A more rigorous follow-on study would deploy the prototype in a

monitored environment equipped with Wazuh, OSSEC, Suricata, and a commercial EDR agent, then measure which indicators each tool fires on under default and tuned configurations. This would produce more actionable guidance for defenders about which detection approaches provide the best coverage-to-false-positive trade-off.

E. ARM Architecture and Embedded Deployments

Raspberry Pi and similar ARM-based single-board computers are increasingly common as inexpensive long-term monitoring platforms in physical security and IoT research contexts. Evaluating the prototype's resource profile on ARM hardware—where CPU performance per watt is lower and memory constraints are tighter—would provide useful data for researchers working in those deployment contexts. PulseAudio initialization behavior on embedded Linux distributions also warrants separate investigation.

F. Bandwidth-Aware Telemetry Compression

Audio artifacts currently dominate transmission payload size at 8.4 megabytes per ten-second clip. Substituting WAV with Opus codec encoding at 32 kbps—achievable through the opuslib Python binding—would reduce audio payload to approximately 40 kilobytes per clip, a 200-fold reduction with acceptable speech intelligibility at that bitrate. This would also change the transmission payload distribution substantially, making screenshots the dominant artifact rather than audio, and might alter the behavioral detection profile of the outbound SMTP sessions.

X. CONCLUSION

A Python-based endpoint telemetry prototype integrating six concurrent data collection modules was designed, built, and evaluated across desktop, laptop, and virtual machine platforms under controlled workloads. The system maintained sub-5 percent average CPU overhead on all physical hardware configurations, with transient spikes reaching 11 to 14 percent during screenshot compression and SMTP transmission on physical hardware and 17 to 22 percent on the virtual machine under elevated load. Memory utilization peaked during screenshot encoding at roughly 2.5 times the steady-state baseline, then returned to baseline within seconds. Keyboard event capture fidelity exceeded 99 percent across input categories; SMTP transmission completed successfully on 98 percent of scheduled cycles.

Several engineering observations emerged that are not commonly discussed in the security literature. PNG compression produces blocking behavior that introduces scheduling drift on resource-constrained platforms. PulseAudio's lazy initialization on Ubuntu 22.04 caused non-deterministic audio capture failures that required workarounds. Webcam initialization latency varied by a factor of five across tested hardware vendors. VM scheduling overhead inflated CPU variance beyond what the application code itself contributed. These are the kinds of operational realities that matter when defenders try to build accurate behavioral heuristics.

From the defensive perspective, the prototype generates several independently detectable behavioral signals: keyboard hook registration visible to hook enumeration APIs, camera device access events logged by OS privacy frameworks, periodic write-encrypt-delete file cycles in temp directories, and highly regular SMTP connection timing that stands out in outbound flow analysis. None of these individually constitutes a definitive alert, but their combination, particularly the network periodicity correlated with CPU spikes and temp-directory activity, provides a detection fingerprint that a well-configured defense stack should surface.

All work was conducted within isolated laboratory environments on researcher-owned hardware. Unauthorized deployment of monitoring software of this type is illegal under applicable laws in virtually all jurisdictions and is categorically outside the intent of this project. The documented behavioral indicators and the resource characterization data are published to assist the defensive security community in building better detection coverage, not to lower the barrier to unauthorized surveillance.

. REFERENCES

[1] A. Bhardwaj and S. Goundar, "Keyloggers: Silent cyber security weapons," Network Security, vol. 2020, no. 2, pp. 14–19, Feb. 2020, doi: 10.1016/S1353-4858(20)30021-0.

[2] L. Zhuang, F. Zhou, and J. D. Tygar, "Keyboard acoustic emanations revisited," ACM Transactions on Information and System Security, vol. 13, no. 1, pp. 1–26, Oct. 2009, doi: 10.1145/1609956.1609959.

[3] R. Kapare, A. Gawade, A. Kamble, and P. Tembhurnikar, "Enhancing digital security with the advanced keylogger project," International Journal of Creative Research Thoughts (IJCRT), vol. 12, no. 3, pp. j617–j624, Mar. 2024.

[4] P. Sinha and M. Yadav, "Keylogger — Capture (keystrokes), screenshot, audio file, operating system information with IP address," B.Tech Project Report, Sathyabama Institute of Science and Technology, Chennai, India, Apr. 2022.

[5] R. Mourya, K. Patil, and Srivaramangai R, "An appraisal of a keylogging and cryptography-based real-time keystroke monitoring system for enhancing cloud security," International Journal of Science and Research (IJSR), vol. 13, no. 4, pp. 65–72, Apr. 2024, doi: 10.21275/SR24331161039.

[6] P. Tuli and P. Sahu, "System monitoring and security using keylogger," International Journal of Computer Science and Mobile Computing, vol. 2, no. 3, pp. 106–111, Mar. 2013.

[7] D. Le, C. Yue, T. Smart, and H. Wang, "Detecting kernel level keyloggers through dynamic taint analysis," College of William & Mary, Dept. of Computer Science, Tech. Rep. WM-CS-2008-05, May 2008.

[8] S. Sagiroglu and G. Canbek, "Keyloggers: Increasing threats to computer security and privacy," IEEE Technology and Society Magazine, vol. 28, no. 3, pp. 10–17, Fall 2009, doi: 10.1109/MTS.2009.934159.

[9] K. Borders and A. Prakash, "Web tap: Detecting covert web communication," in Proc. ACM CCS, Washington DC, USA, 2004, pp. 110–120, doi: 10.1145/1030083.1030099.

[10] G. Hoglund and J. Butler, Rootkits: Subverting the Windows Kernel. Reading, MA: Addison-Wesley Professional, 2006.

[11] D. Arp et al., "DREBIN: Effective and explainable detection of Android malware in your pocket," in Proc. NDSS, San Diego, CA, USA, 2014, pp. 1–15.

[12] OSSEC Open Source HIDS SECurity, "OSSEC project documentation," [Online]. Available: https://www.ossec.net/

[13] Wazuh, Inc., "Wazuh unified XDR and SIEM documentation," [Online]. Available: https://documentation.wazuh.com/

[14] N. Provos and T. Holz, Virtual Honeypots: From Botnet Tracking to Intrusion Detection. Reading, MA: Addison-Wesley Professional, 2007.

[15] A. Wajahat, A. Imran, J. Latif, A. Nazir, and A. Bilal, "A novel approach of unprivileged keyloggers detection," in Proc. 2nd IEEE Int. Conf. Computing, Mathematics and Engineering Technologies (iCoMET), Sukkur, Pakistan, 2019, doi: 10.1109/ICOMET.2019.8673404.